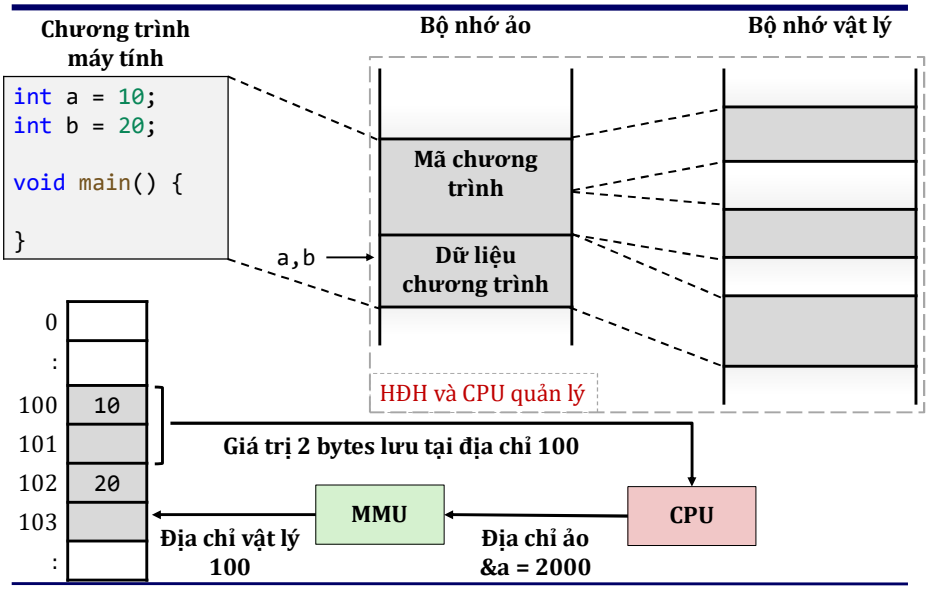
Bài 8

Ngôn ngữ C
Con trỏ

Nội dung

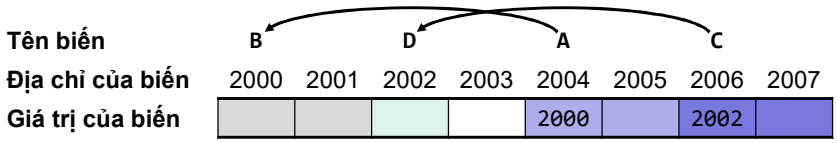
- Địa chỉ bộ nhớ
- Con trỏ
- Các phép toán với con trỏ
- Con trỏ trỏ tới con trỏ
- Lỗi phổ biến với con trỏ

Địa chỉ bộ nhớ



Con trỏ

- **Con trỏ** là một loại biến dùng để ***lưu trữ địa chỉ bộ nhớ*** của một biến khác.
- Khi biến A lưu trữ địa chỉ của biến B, ta nói A là một con trỏ trỏ (tham chiếu) tới B.



- Con trỏ có thể dùng để duyệt mảng, truyền đối số cho hàm, cấp phát bộ nhớ động.

Con trỏ - Khai báo

- Khai báo con trỏ

  *<kiểu> *<tên biến>;*

- Trong đó
  - *<kiểu>* của con trỏ xác định kiểu của đối tượng (biến) mà con trỏ sẽ trỏ tới.
  - Dấu * để cho biết *<tên biến>* là một con trỏ.

```
/* px, py là các con trỏ kiểu int.
   Chúng sẽ được dùng để lưu trữ
   địa chỉ của các biến kiểu int. */
int *px, *py;
```


Con trỏ - Khởi tạo

- Khởi tạo con trỏ

  *<kiểu> *<tên biến> = <địa chỉ của biến khác>;*

- Không như biến thông thường, theo qui ước các con trỏ chưa được xác định trỏ tới một biến cụ thể nên được khởi tạo bằng địa chỉ **NULL** (bằng 0).

```
int x;
/* khởi tạo con trỏ px trỏ tới x.
   Phép toán & để lấy địa chỉ biến x. */
int *px = &x;
// khởi tạo con trỏ py, pz bằng địa chỉ NULL.
int *py = NULL;
int *pz = 0;
```

Các phép toán với con trỏ

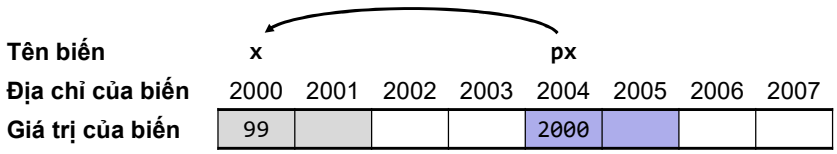
- Phép toán địa chỉ.
- Phép toán truy cập gián tiếp.
- Phép gán con trỏ.
- Phép toán ép kiểu con trỏ.
- Phép toán cộng, trừ địa chỉ.
- Các phép toán quan hệ.

Phép toán lấy địa chỉ của biến

- **Phép toán địa chỉ &** (address-of operator / reference operator) dùng để lấy ***địa chỉ của một biến*** trên bộ nhớ

  &<biến>

```
int x = 99, *px = NULL;
/* gán địa chỉ của x cho con trỏ px. Khi đó px
   trỏ (tham chiếu) tới x. Phép toán địa chỉ &
   còn được gọi là phép toán tham chiếu. */
px = &x;
```



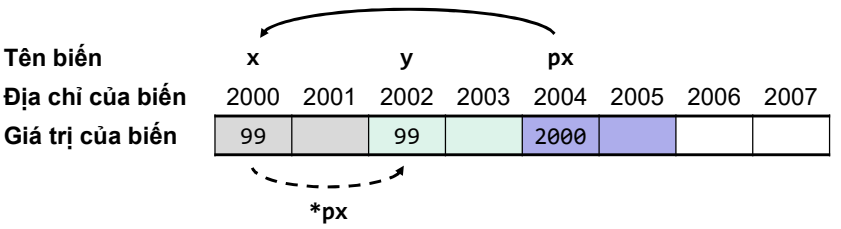
Phép toán truy cập gián tiếp (1)

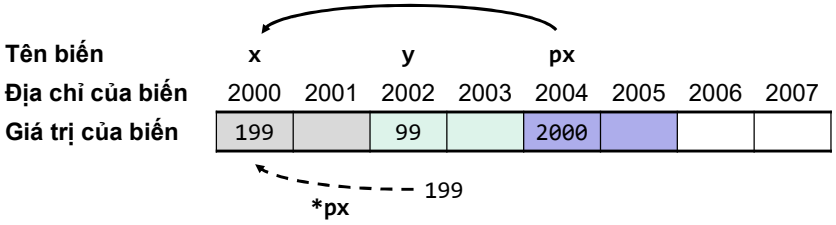
- **Phép toán truy cập gián tiếp *** (indirection operator / dereference operator) dùng để truy cập ***giá trị của một biến*** thông qua một con trỏ trỏ tới nó

  **<con trỏ>*

```
int x = 99, y, *px = NULL;
// con trỏ px trỏ tới x.
px = &x;
// lấy giá trị của x thông qua px rồi gán cho y.
y = *px;
// gán 199 cho x thông qua px.
*px = 199;
```


Phép toán truy cập gián tiếp (2)



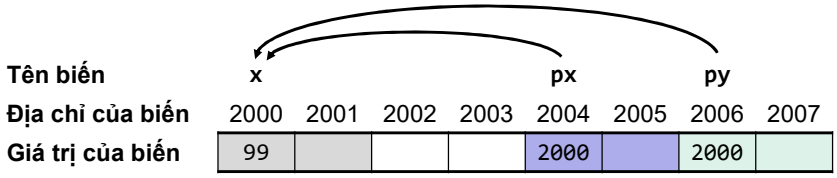


Phép gán con trỏ

- Phép gán dùng để gán giá trị của một con trỏ cho một con trỏ cùng kiểu khác

  <con trỏ> = <con trỏ>

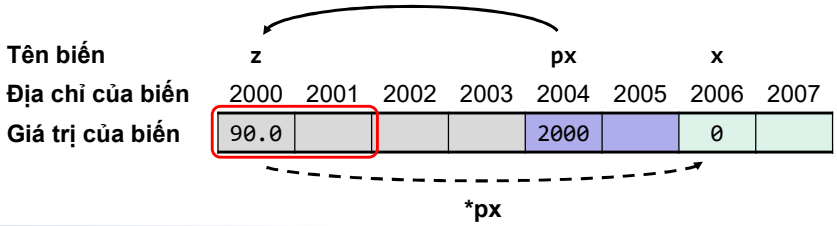
```
int x = 99, *px = NULL, *py = NULL;
// con trỏ px trỏ tới x.
px = &x;
// con trỏ py cũng trỏ tới x.
py = px;
```



Phép toán ép kiểu con trỏ

- Phép toán ép kiểu để ép một con trỏ trỏ tới một biến không cùng kiểu với nó, có thể dẫn đến hành vi không xác định.

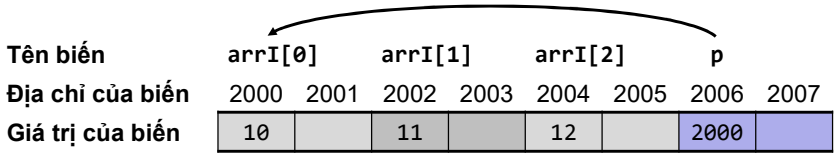
```
double z = 90.0;
int x, *px = NULL;
px = (int *) &z; // ép con trỏ px trỏ tới z.
// lấy giá trị của z thông qua px rồi gán cho x.
x = *px;
```



Phép cộng, trừ địa chỉ (1)

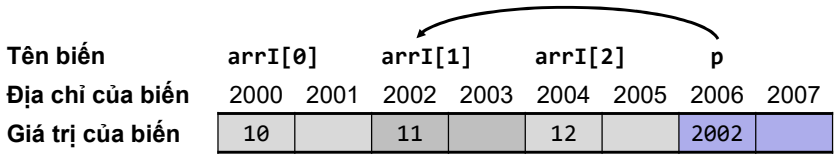
- **Phép cộng/trừ con trỏ với một số nguyên** dùng để tăng/giảm địa chỉ lưu trữ trong con trỏ.
  - Điều này cho phép con trỏ tới vị trí bộ nhớ khác, nghĩa là trỏ tới một đối tượng khác.
  - **Thường được sử dụng để duyệt các mảng.**

```
int arrI[3] = {10, 11, 12};
int *p = NULL;
// con trỏ p trỏ tới phần tử arrI[0].
p = &arrI[0];
```



Phép cộng, trừ địa chỉ (2)

```
/* giá trị của p tăng lên 1 đơn vị kích thước
   kiểu của arrI[0] (tăng lên 1 * sizeof(int)),
   nghĩa là p trỏ tới arrI[1]. */
p = p + 1;
```



- Phép ++, -- cũng có thể sử dụng với con trỏ với ý nghĩa tương tự.

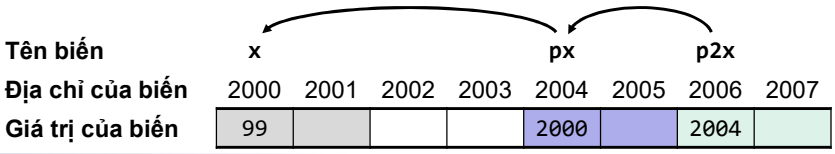
Phép toán quan hệ

- Các phép toán <, <=, >, >=, ==, != dùng để so sánh địa chỉ chứa trong 2 con trỏ
  - Hai con trỏ là bằng nhau nếu cùng trỏ tới một vị trí bộ nhớ (cùng trỏ tới một đối tượng).
  - Ngược lại, con trỏ nào chứa giá trị địa chỉ lớn hơn thì lớn hơn.
- Chỉ có ý nghĩa sử dụng khi
  - So sánh một con trỏ với địa chỉ NULL.
  - So sánh 2 con trỏ trỏ tới một đối tượng chung như mảng.

Con trỏ trỏ tới con trỏ (1)

- Một con trỏ có thể trỏ tới một con trỏ khác đang trỏ tới một biến. Trường hợp này được gọi là **con trỏ trỏ tới con trỏ** hay **tác động gián tiếp bội** (multiple indirection).
- Để khai báo một con trỏ trỏ tới một con trỏ khác ta thêm một dấu * phía trước tên biến.

```
int x = 99, y;
int *px = &x;      // px trỏ tới x.
int **p2x = &px;   // p2x trỏ tới px.
```



Con trỏ trỏ tới con trỏ (2)

- Để truy cập giá trị của biến đích thông qua một con trỏ nhiều mức tác động gián tiếp tới nó, ta sử dụng phép toán truy cập gián tiếp với số dấu * tương ứng với số mức của con trỏ.

```
/* lấy giá trị của x thông qua p2x rồi gán
   cho y. */
y = **p2x;
// gán 100 cho x thông qua p2x.
**p2x = 100;
```

- Tác động gián tiếp hiếm khi sử dụng quá 2 mức. Khi quá nhiều mức rất khó theo dõi và dễ mắc phải các lỗi về khái niệm.

Lỗi phổ biến với con trỏ

- **Con trỏ NULL** hoặc **con trỏ chưa khởi tạo là các con trỏ chưa hợp lệ để sử dụng.** Nếu dùng để lưu giá trị vào vị trí bộ nhớ có thể gây ra lỗi nghiệm trọng cho chương trình.

```
int x = 99, *px, *py = NULL;
// py là con trỏ NULL, chưa hợp lệ để truy cập.
*py = x;
// px chưa khởi tạo, chưa hợp lệ để truy cập.
*px = 99;
```

- **Gán một giá trị cho con trỏ thay vì địa chỉ.**

```
int x = 99, *px; // x là giá trị kiểu int.
px = x;          // x không phải là địa chỉ.
```